

Real-Time Multisensor Room Monitoring System

Course: BCA152 Microcontrollers | Department of Computer Applications — MSU-IIT

Author: Shyane Novelle R. Canayan

Instructor: Asst. Prof. Paul Rodolf P. Castor, M.Sc.

Date: September 2026

I. INTRODUCTION

1.1 Problem Context and Statement

Smart facility monitoring systems need to collect sensor data reliably, monitor safety conditions continuously, and respond to user input without delay. Conventional Arduino programs, however, rely on a single sequential loop (`void loop()`) to handle every task one after another. When this loop includes blocking delays for sensor readings, the whole system stalls—making the interface sluggish and causing inconsistent timing.

This project addresses that problem by building a real-time room monitoring system on the ESP32 microcontroller. Instead of a single sequential loop, the system uses FreeRTOS, a real-time operating system built into the Espressif IoT Development Framework (ESP-IDF), to run multiple tasks concurrently and preemptively.

1.2 System and Functional Requirements.

The firmware was designed to satisfy ten mandatory functional requirements, grouped into four categories:

FR-01 to FR-03 (Telemetry Acquisition): The system shall perform periodic, drift-free sampling of temperature, relative humidity, and ambient light intensity. Light intensity is acquired through single-shot ADC (Analog-to-Digital Converter) conversion, while temperature and humidity are acquired through software-timed, single-wire digital communication with the sensor.

FR-04 to FR-06 (User Interaction): The system shall detect human presence via a PIR sensor and support cyclic navigation between display pages (Temperature, Humidity, Light, and Motion) using a quadrature rotary encoder interfaced with an SSD1306 OLED display.

FR-07 (Acoustic Safety Alarm): The system shall continuously evaluate the sampled temperature against defined upper and lower setpoint thresholds and activate a hardware buzzer when either threshold is exceeded.

FR-08 to FR-10 (Power State Management): The system shall implement a two-state finite state machine (FSM) that automatically transitions from ACTIVE to INACTIVE following a 15-second period of undetected motion, and restores the ACTIVE state immediately upon subsequent motion detection.

II. METHODOLOGY

2.1 System Architecture and Hardware Decomposition

The system architecture follows a clean 4-tier model:

1. **Physical Hardware Abstraction Layer.** This layer connects directly to the simulated ESP32 hardware, which includes the DHT22 sensor, an LDR connected to ADC1_CH6, an HC-SR501 PIR motion sensor, an EC11 rotary encoder, an SSD1306 OLED display, and an active buzzer.
2. **Driver Layer.** This layer contains the low-level code for each device, such as manual signal timing, ADC attenuation settings, and I2C communication. This code is organized into separate files for each function (sensors.cpp, display.cpp, input.cpp, motion.cpp, alarm.cpp).
3. **Kernel and OS Services Layer.** This layer manages task scheduling, divides work between the two CPU cores, and handles safe communication between tasks using FreeRTOS (rtos_objects.cpp).
4. **Application Logic Layer.** This layer contains the system's decision-making functions (system_state.cpp). It is kept separate from the hardware code so it can be tested on a computer without needing the actual ESP32 board.

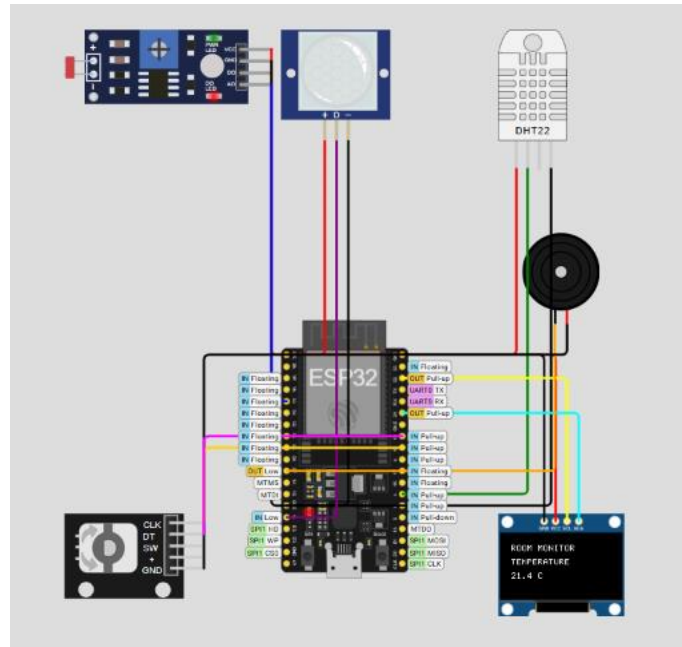


Figure 1. Wokwi circuit diagram of the complete room monitoring system.

2.2 FreeRTOS Task Architecture and Scheduling

Table 1: Required FreeRTOS Task Table

Task Name	Responsibility	Trigger/Period	Priority	Core	IPC Mechanism	Typical Blocked Condition
MotionTask	Sample PIR; manage inactivity timeout	Periodic (50 ms)	3	Core 0	systemEvents (Event Group)	vTaskDelay
InputTask	Decode rotary encoder gray code	Periodic (20 ms)	3	Core 0	modeQueue	vTaskDelay
SensorTask	Acquire DHT22 and LDR	Periodic (2000 ms)	2	Core 1	sensorQueue	vTaskDelayUntil

	measurements					
AlarmTask	Evaluate thermal bounds; drive buzzer	Event (Queue update)	2	Core 1	sensorQueue	xQueueReceive (Block)
DisplayTask	Render UI pages on OLED	Event (Telemetry/Mode)	1	Core 1	sensorQueue, modeQueue	xQueueReceive (Block)

FreeRTOS Task States Dynamics

In this architecture, tasks move predictably between the FreeRTOS scheduler states:

Blocked: Most tasks spend the majority of their time in the Blocked state, giving up the CPU while waiting. MotionTask and InputTask block using vTaskDelay(), SensorTask blocks using vTaskDelayUntil(), and DisplayTask and AlarmTask block indefinitely while waiting for new data through xQueueReceive().

Ready: A task moves to the Ready state once its delay period ends or new sensor data becomes available in sensorQueue. At this point, the task waits for the scheduler to assign it CPU time based on priority.

Running: The FreeRTOS scheduler selects the highest-priority Ready task to run on its assigned Core 0 for user input handling, and Core 1 for sensor reading, alarm evaluation, and display rendering and immediately taking over from any lower-priority task currently running.

2.3 Technical Priority Justification

Priority 3 (InputTask, MotionTask): Encoder turns and motion detection are real-world events that happen suddenly and briefly. If these tasks were given a lower priority, encoder inputs could be missed and the UI would feel slow to respond. Priority 3 ensures these tasks immediately interrupt background display rendering before entering their short waiting periods.

Priority 2 (SensorTask, AlarmTask): Reading sensor data and checking alarm conditions are safety-critical operations. Once new sensor data is available, AlarmTask interrupts lower-priority rendering tasks to turn the buzzer (GPIO 23) on or off without delay.

Priority 1 (DisplayTask): Sending 1024-byte display buffers over I2C at 400 kHz takes a relatively long time to complete. Keeping DisplayTask at the lowest priority prevents display updates from blocking or delaying.

2.4 Inter-Task Communication and Synchronization

Queues: `sensorQueue` transmits `SensorData` structs using `xQueueOverwrite()` with a queue depth of 1. This design guarantees that both the display and alarm tasks always access the latest available sensor reading, eliminating the need to buffer historical data.

Mutexes: `serialMutex` synchronizes UART logging between Core 0 and Core 1, ensuring only one core writes to the serial terminal at a time. Omitting this safeguard would result in overlapping log outputs that are difficult to read.

Event Group: `systemEvents` represents the system's operating state through dedicated bit flags: `EVENT_ACTIVE` (BIT0), `EVENT_MOTION` (BIT1), and `EVENT_ALARM` (BIT2).

Deterministic Periodic Execution: `SensorTask` relies on `vTaskDelayUntil()` rather than `vTaskDelay()` to maintain consistent execution timing. Because this function schedules based on absolute time rather than elapsed time since the last call, variations in sensor processing duration do not accumulate into timing drift.

2.5 State Machine Design

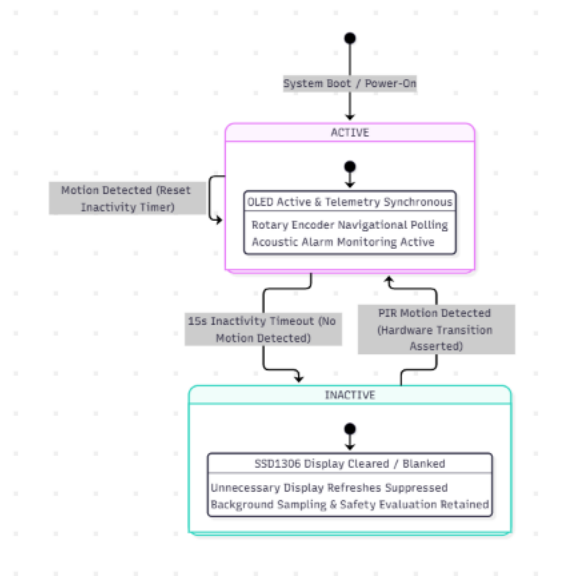


Figure 2. *Two-state embedded finite state machine (FSM) governing ACTIVE and INACTIVE operational modes.*

ACTIVE State: This state begins immediately when the PIR sensor detects movement. In this state, the SSD1306 OLED display stays on, sensor readings update in real time, and the rotary encoder can be used to switch between display pages.

INACTIVE State: The system automatically enters this state after 15 seconds without detecting motion (FT-09). The OLED screen is cleared and turned off to save power and prevent screen burn-in, while sensor readings and temperature safety checks continue running in the background.

2.6 Implementation Decisions

Keeping peripheral driver code separate from the decision-making logic made it possible to run automated tests without needing physical hardware or hardware simulators. Assigning DisplayTask as the single owner of the SSD1306 OLED display avoided conflicts on the I2C bus, without the need for complicated, nested mutex locks.

In addition, using `xQueueOverwrite()` instead of a large FIFO buffer avoided unnecessary memory usage, while ensuring that consumer tasks always process the most recent room environment readings.

III. RESULTS

3.1 Unit Testing Results

A total of thirteen discrete unit tests were implemented in the `test/` directory and executed off-target using `pio test`, all of which passed:

- 1. Temperature Evaluation (5 tests):** Verified correct classification at five boundary conditions, confirming accurate transitions between `LOW_TEMPERATURE`, `NORMAL`, and `HIGH_TEMPERATURE` states at the defined threshold values.
- 2. Rotary Navigation (4 tests):** Confirmed correct display page cycling in both directions, including forward sequential navigation, forward wraparound (from Motion back to Temperature), reverse sequential navigation, and reverse wraparound (from Temperature back to Motion).
- 3. State Machine Transitions (4 tests):** Verified correct finite state machine behavior, including sustained `ACTIVE` state during continued motion, automatic transition to `INACTIVE` after a 15-second motion timeout, persistence of the `INACTIVE` state in the absence of motion, and immediate reactivation to `ACTIVE` upon motion detection.

3.2 Functional Verification Record in Wokwi

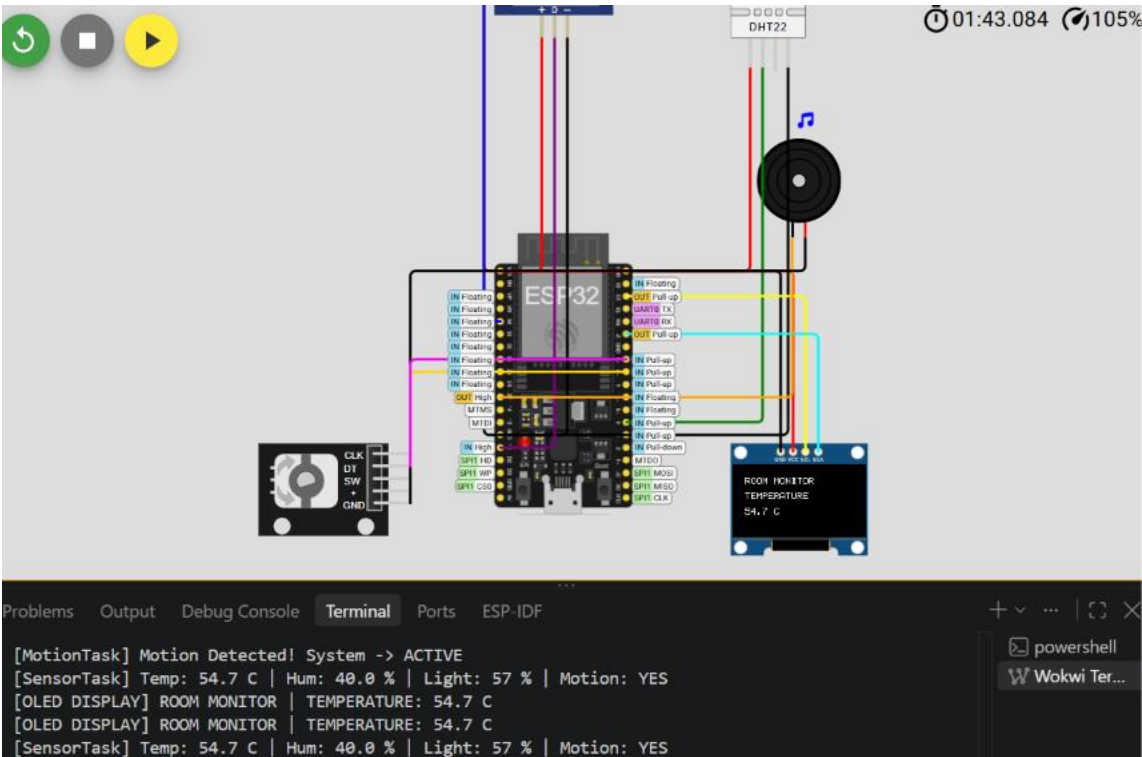


Figure 3. OLED display output showing real-time temperature reading.

All 10 required end-to-end integration scenarios were simulated and validated on Wokwi.

Table 2: Functional Verification Record

Test ID	Input / Stimulus	Expected Behavior	Actual Observed Output	Status
FT-01	Temperature changed to 54.7°C	Displayed temperature updates	OLED displays 54.7°C synchronously	PASS
FT-02	Humidity slider changed to 65.5%	Displayed humidity updates	OLED displays 65.5% synchronously	PASS

FT-03	LDR illumination adjusted to 38%	Displayed light value updates	OLED displays Light: 38%	PASS
FT-04	Rotary encoder rotated CW	Page advances sequentially	Mode advances: TEMP → HUM → LIGHT → MOTION	PASS
FT-05	Rotary encoder rotated CCW	Page regresses sequentially	Mode decrements in reverse order	PASS
FT-06	Temperature set to 54.7°C (> 30°C)	Acoustic alarm activates	Buzzer pin GPIO 23 asserts HIGH; alarm state triggers	PASS
FT-07	Temperature returned to 24.0°C	Acoustic alarm stops	Buzzer pin GPIO 23 de-asserts LOW; normal state restored	PASS
FT-08	Motion simulated on PIR sensor	System enters/maintains ACTIVE	System remains in ACTIVE; OLED display stays enabled	PASS
FT-09	15 seconds elapsed without motion	System enters INACTIVE state	Inactivity timeout triggered; OLED clears display	PASS

FT-10	Motion triggered during INACTIVE	System returns to ACTIVE	Motion interrupt detected; display and navigation reactivate	PASS
-------	----------------------------------	--------------------------	--	------

3.3 Deliberate Fault Experiments

- **Fault Experiment 1 (Remove Blocking Delay):** When `vTaskDelayUntil()` was removed from `SensorTask`, the task entered an uncontrolled loop with no pause between executions. Since `SensorTask` ran at Priority 2 on Core 1, it consumed all available CPU cycles on that core, completely blocking `DisplayTask` (Priority 1) from running and causing the OLED display to freeze.
- **Fault Experiment 2 (Unbalanced Task Priority):** Raising `DisplayTask` to Priority 4 caused the encoder to become less responsive. While the screen was redrawing, rotating the encoder resulted in missed input signals, since `InputTask` was denied sufficient CPU time.
- **Fault Experiment 3 (Remove Serial Mutex):** When `serialMutex` was disabled, simultaneous logging from `SensorTask` and `MotionTask` produced garbled, overlapping output on the serial terminal (e.g., `[SensorTask] Te[MotionTask] Inactivity mp: 25.4C`).

3.4 Static Code Analysis Findings

Running PlatformIO Cppcheck (pio check) revealed three portability notices:

Table 3: Static Analysis Findings Table

Finding	File / Location	Root Cause	Engineering Resolution
[low:portability] memset() on struct with float	src/alarm.cpp:28	Cppcheck flags memset() on floating-point members as theoretically non-	Analyzed and Accepted: The ESP32 Xtensa architecture strictly adheres to IEEE 754 single-

		portable across non-IEEE architectures.	precision representation. Zero-initialization via memset() introduces no runtime or logical defects, though zero-initialization syntax (SensorData data = {};
[low:portability] memset() on struct with float	src/display.cpp:221	Same IEEE 754 zero-bit pattern check on display cache structure.	Analyzed and Accepted: Verified safe for the Xtensa core target. The operation incurs negligible execution overhead and causes no undefined behavior or memory corruption during task execution
[low:portability] memset() on struct with float	src/sensors.cpp:104	Same IEEE 754 zero-bit pattern check on sensor data packet.	Analyzed and Accepted: Confirmed on the target platform. Memory layout remains valid across all builds, with zero impact on numerical calculations or sensor queue operations

```

PS C:\Users\USER\Documents\bca152-freertos-multisensor> pio check
Checking esp32dev > cppcheck (platform: espressif32; board: esp32dev; framework: espidf)
-----
src\alarm.cpp:28: [low:portability] Using memset() on struct which contains a floating point number. [memsetClassFloat]
src\display.cpp:221: [low:portability] Using memset() on struct which contains a floating point number. [memsetClassFloat]
src\sensors.cpp:104: [low:portability] Using memset() on struct which contains a floating point number. [memsetClassFloat]
===== [PASSED] Took 4.31 seconds =====
Component      HIGH      MEDIUM    LOW
-----
src             0         0         3
Total          0         0         3

Environment    Tool      Status    Duration
-----
esp32dev       cppcheck  PASSED    00:00:04.310
===== 1 succeeded in 00:00:04.310 =====
PS C:\Users\USER\Documents\bca152-freertos-multisensor>

```

Figure 4. PlatformIO Cppcheck output showing static analysis findings.

3.5 Requirements Traceability Matrix

The following matrix confirms that all specified functional requirements are implemented in modular source code and verified by empirical evidence:

Requirement ID	Description	Source Module Implementation	Verification Evidence
FR-01, FR-02	DHT22 Periodic Sampling	src/sensors.cpp (SensorTask)	FT-01, FT-02
FR-03	Ambient Light (LDR ADC)	src/sensors.cpp (read_ldr_scaled)	FT-03
FR-04	PIR Motion Detection	src/motion.cpp (MotionTask)	FT-08
FR-05	SSD1306 Dynamic Telemetry Display	src/display.cpp (DisplayTask)	FT-01, FT-02, FT-03
FR-06	Rotary Encoder Page Traversal	src/input.cpp (InputTask)	Unit Navigation Tests + FT-04, FT-05
FR-07	Temperature Bounds Buzzer Alarm	src/alarm.cpp (AlarmTask)	Unit Alarm Tests + FT-06, FT-07

FR-08	Multi-state Operation	src/system_state.cpp	Unit State Tests
FR-09	15s Inactivity Auto-Sleep	src/motion.cpp, src/system_state.cpp	Unit State Tests + FT-09
FR-10	Automatic Motion Wake	src/motion.cpp, src/system_state.cpp	Unit State Tests + FT-10

IV. DISCUSSION

4.1 Engineering Trade-offs and Design Decisions

Periodic Polling vs. Hardware Interrupts: The rotary encoder is checked every 20 ms and the PIR sensor every 50 ms using high-priority FreeRTOS tasks, instead of using hardware interrupts. Interrupts can respond faster, but they can also trigger repeatedly and unpredictably due to the natural bouncing of the encoder's mechanical switch. Checking the sensors at fixed intervals avoids this problem while still keeping CPU usage low and predictable.

Overwriting Queue vs. Deep FIFO Queue: Using `xQueueOverwrite()` with a queue depth of 1 ensures the display and safety checks always respond immediately to the latest data. For room monitoring, older readings do not matter — only the current condition of the room is important for making decisions.

4.2 Debugging and Challenges Encountered

During firmware integration on the dual-core system, two main challenges were encountered:

Multi-Core UART Collision: When `SensorTask` on Core 1 and `MotionTask` on Core 0 attempted to log data at the same time, the console output became broken and mixed together. This was resolved by protecting all UART output with a dedicated FreeRTOS mutex (`serialMutex`), ensuring only one task writes to the terminal at a time.

I2C Display Bus Contention: Allowing multiple tasks to write to or clear the OLED screen caused the I2C bus to lock up. This was resolved by making DisplayTask the only task allowed to control the SSD1306 display, which completely eliminated this conflict.

4.3 Practical Limitations

1. **Simulated vs. Physical ADC:** The Wokwi simulator produces linear, idealized readings from the light sensor on ADC1 Channel 6 (GPIO 34). Actual ESP32 hardware, however, loses accuracy at very low (below 0.1V) and very high (above 3.2V) input voltages, meaning real-world deployment would require software calibration to maintain reading precision.
2. **PIR Sensor Warm-Up Time:** Physical HC-SR501 PIR sensors typically require 30 to 60 seconds after power-up to stabilize before accurately detecting motion. The Wokwi simulation does not model this warm-up period and instead responds to motion immediately.
3. **Fixed Inactivity Window:** The 15-second inactivity timeout was suitable for rapid testing during laboratory trials (FT-09). In practical use, however, this duration is too short, since a person remaining still in the room would cause the display to repeatedly turn off and back on.

4.4 Future Improvements

1. **Adjustable Motion Timeout:** Extend the default display timeout to a range of 3–5 minutes, with the option for users to customize this duration via the rotary encoder. This would prevent the screen from repeatedly switching off while the room remains occupied.
2. **Improved Presence Detection:** Replace or supplement the PIR sensor with an mmWave presence sensor (such as the LD2410), which is capable of detecting stationary occupants through minor movements such as breathing.
3. **Persistent Settings via Non-Volatile Storage (NVS):** Utilize the ESP32's built-in NVS flash storage to retain user-defined settings including the timeout duration and temperature alarm thresholds (18°C to 30°C) across power cycles and restarts.
4. **Cloud Dashboard Integration:** Implement a dedicated Wi-Fi task to transmit sensor data to a remote MQTT-based dashboard, enabling real-time monitoring from an external location.

5. **Power-Efficient Sleep Mode:** Configure the ESP32 to enter Deep Sleep mode during extended periods of inactivity, with the PIR sensor (GPIO 13) configured as a wake-up trigger upon motion detection.

V. Conclusion

This project successfully met all the requirements of Laboratory Activity No. 1. By using FreeRTOS with ESP-IDF, the system was able to run multiple tasks at the same time without conflicts, take sensor readings on schedule, and respond quickly to user input. This showed a clear improvement over the traditional single-loop approach usually used in basic Arduino programs. Through this activity, I learned how to apply real-time operating system concepts such as task priorities, queues, event groups, and mutexes to manage multiple processes running at once. I also learned the importance of separating hardware code from the main program logic, since this made the system easier to test and debug.

However, the project also has some limitations that can still be improved. For example, the ADC readings would need calibration for real hardware use, the PIR sensor's warm-up time was not considered, and the fixed 15-second inactivity timeout is too short for practical, everyday use. Future improvements could include saving user settings even after restart, adding remote monitoring through Wi-Fi, and adding a sleep mode to save power. Overall, the system worked as intended and proved that using FreeRTOS makes IoT monitoring systems more reliable and efficient, with room for further improvement to make it more practical for real-world use.